

8 位定点 CNN 基本运算模块设计与功能仿真报告

1 引言

本报告旨在完成面向边缘端 AI 的 8 位定点 CNN 基本运算模块设计，包括 8 位无符号乘法器、8 位加法器、ReLU 激活模块和除以 4 除法器，并通过功能仿真验证各模块的正确性，为后续卷积-激活-池化全链路设计奠定基础。

2 各模块设计与实现

2.1 8 位无符号乘法器（卷积乘算）

2.1.1 功能说明

该模块用于卷积运算中的像素值与卷积核权重相乘，输入为两个 8 位无符号数，输出为 16 位无符号乘积，以防止乘法溢出。

2.1.2 Verilog 代码实现

```
1 // mul_8bit.v
2 module mul_8bit (
3     input wire [7:0] a,      // 乘数A (8位无符号)
4     input wire [7:0] b,      // 乘数B (8位无符号)
5     output wire [15:0] product // 乘积 (16位无符号)
6 );
7     assign product = a * b;    // 直接使用Verilog乘法运算符
8 endmodule
```

Listing 1: mul_8bit.v

2.2 8 位加法器（卷积累加、池化求和）

2.2.1 功能说明

该模块用于卷积结果的累加和池化运算中的求和，输入为两个 8 位无符号数，输出为 9 位无符号和，通过扩展位宽防止加法溢出。

2.2.2 Verilog 代码实现

```
1 // add_8bit.v
2 module add_8bit (
```

```

3   input wire [7:0] a,      // 加数A (8位无符号)
4   input wire [7:0] b,      // 加数B (8位无符号)
5   output wire [8:0] sum     // 和 (9位无符号, 防止溢出)
6 );
7   assign sum = a + b;      // 直接使用Verilog加法运算符
8 endmodule

```

Listing 2: add_8bit.v

2.3 ReLU 激活模块（无符号数直接输出）

2.3.1 功能说明

该模块用于 ReLU 激活函数，由于输入为 8 位无符号数（范围 0-255），本身非负，因此无需进行负数判断，直接将输入赋值给输出即可。

2.3.2 Verilog 代码实现

```

1 // relu.v
2 module relu (
3     input wire [7:0] in,      // 输入 (8位无符号)
4     output wire [7:0] out     // 输出 (等于输入, 因无符号数非负)
5 );
6     assign out = in;          // 无符号数无需判断, 直接输出
7 endmodule

```

Listing 3: relu.v

2.4 除以 4 除法器（池化平均）

2.4.1 功能说明

该模块用于 2×2 平均池化运算，输入为 10 位无符号数（兼容 4 个 8 位数累加结果，最大值为 $4 \times 255 = 1020$ ），通过无符号数右移 2 位实现除以 4 取整，输出为 8 位无符号结果。

2.4.2 Verilog 代码实现

```

1 // div4.v
2 module div4 (
3     input wire [9:0] in,      // 输入 (10位, 兼容4个8位数累加结果)
4     output wire [7:0] out     // 输出 (8位, 除以4后结果)
5 );
6     assign out = in >> 2;     // 无符号数右移2位, 等价于除以4取整
7 endmodule

```

Listing 4: div4.v

3 功能仿真验证

3.1 仿真激励代码

编写 Testbench 对四个模块进行统一测试，覆盖典型值和边界值（如 0、255 等）。

```
1 // tb_top.v
2 `timescale 1ns/1ps // 时间单位/精度
3
4 module tb_top;
5     // 乘法器测试信号
6     reg [7:0] mul_a, mul_b;
7     wire [15:0] mul_product;
8
9     // 加法器测试信号
10    reg [7:0] add_a, add_b;
11    wire [8:0] add_sum;
12
13    // ReLU测试信号
14    reg [7:0] relu_in;
15    wire [7:0] relu_out;
16
17    // 除法器测试信号
18    reg [9:0] div4_in;
19    wire [7:0] div4_out;
20
21    // 实例化所有模块
22    mul_8bit u_mul (
23        .a(mul_a),
24        .b(mul_b),
25        .product(mul_product)
26    );
27
28    add_8bit u_add (
29        .a(add_a),
30        .b(add_b),
31        .sum(add_sum)
32    );
33
34    relu u_relu (
35        .in(relu_in),
36        .out(relu_out)
37    );
38
39    div4 u_div4 (
40        .in(div4_in),
41        .out(div4_out)
42    );
43
44    // 测试激励
```

```

45  initial begin
46      // 初始化输入
47      mul_a = 0; mul_b = 0;
48      add_a = 0; add_b = 0;
49      relu_in = 0;
50      div4_in = 0;
51      #10;
52
53      // -----
54      // 测试1: 乘法器
55      // -----
56      mul_a = 8'd2; mul_b = 8'd3; #10; // 2*3=6
57      mul_a = 8'd255; mul_b = 8'd255; #10; // 255*255=65025
58
59      // -----
60      // 测试2: 加法器
61      // -----
62      add_a = 8'd100; add_b = 8'd200; #10; // 100+200=300
63      add_a = 8'd255; add_b = 8'd255; #10; // 255+255=510
64
65      // -----
66      // 测试3: ReLU
67      // -----
68      relu_in = 8'd123; #10; // 输入123, 输出123
69      relu_in = 8'd0; #10; // 输入0, 输出0
70
71      // -----
72      // 测试4: 除以4
73      // -----
74      div4_in = 10'd1020; #10; // 1020/4=255
75      div4_in = 10'd100; #10; // 100/4=25
76
77      #10;
78      $finish; // 结束仿真
79  end
80 endmodule

```

Listing 5: tb_top.v

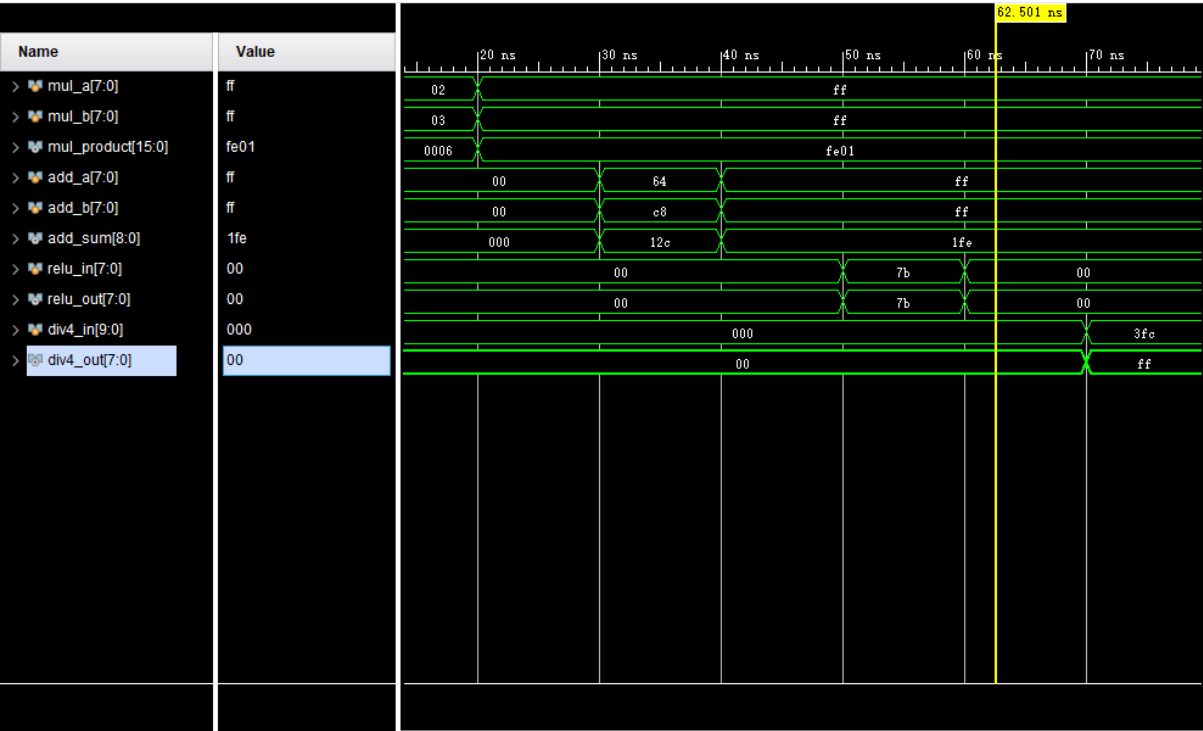


图 1: 功能仿真波形图

4 结论

本报告完成了 8 位无符号乘法器、8 位加法器、ReLU 激活模块和除以 4 除法器的设计与实现，并通过功能仿真验证了各模块的逻辑正确性。仿真结果表明，所有模块均能正常工作，覆盖了典型值和边界值测试，可用于后续卷积-激活-池化全链路的设计与集成。